

LAB REPORT
ON
OBJECT ORIENTED PROGRAMMING WITH C++
Phase I

By
Dipak Bahadur Chand
Exam Roll No:
TU Regd. No: 9-2-29-6-2025



Submitted to:
Piyush Pant
Mahendra Ratna Campus

In partial fulfillment of the requirements for the Course
Object Oriented Programming with C++

Tahachal, Kathmandu
September 2026

Table of Contents

1	Write a C++ program to input a number and check whether it is a prime number or not.	1
1.1	Source Code.	1
1.2	Output Window.	2
2	Write a C++ program to find the sum of numbers from 1 to n.	2
2.1	Source Code.	2
2.2	Output Window.	3
3	Write a C++ program to find the factorial of a number using recursion.	3
3.1	Source Code.	3
3.2	Output Window.	4
4	Write a C++ program to calculate the area of a circle, rectangle, and square using method overloading.	4
4.1	Source code.	4
4.2	Output Window.	6
5	Write a C++ program to swap two numbers using pass by value.	6
5.1	Source Code.	6
5.2	Output Window.	7
6	Write a C++ program to swap two numbers using pass by reference.	8
6.1	Source Code.	8
6.2	Output Window.	9
7	Write a C++ program to swap two numbers without using a third variable.	9
7.1	Source Code.	9
7.2	Output Window.	10
8	Write a C++ program to input a number and print its reverse.	10
8.1	Source Code.	10
8.2	Output Window.	11
9	Write a C++ program to demonstrate constructor overloading.	11
9.1	Source Code.	11
9.2	Output window.	13
10	Write a C++ program to demonstrate encapsulation.	13
10.1	Source Code.	13
10.2	Output window.	14
11	Write a C++ program to demonstrate a friend function.	14
11.1	Source Code.	14

11.2	Output Window.	16
12	Write a C++ program to demonstrate a static data member.	16
12.1	Source Code.	16
12.2	Output Window.	17
13	Write a C++ program to demonstrate the execution sequence of constructors in base and derived classes.	18
13.1	Source code.	18
13.2	Output Window.	19
14	Write a C++ program to demonstrate different modes of inheritance.	19
14.1	Source Code.	19
14.2	Output Window.	21
15	Write a C++ program to demonstrate overriding methods.	21
15.1	Source Code.	21
15.2	Output Window.	22
16	Write a C++ program to demonstrate arithmetic operator overloading.	22
16.1	Source Code.	22
16.2	Output Window.	23
17	Write a C++ program to demonstrate relational operator overloading.	24
17.1	Source Code.	24
17.2	Output Window.	25
18	Write a C++ program to demonstrate unary operator overloading.	25
18.1	Source Code.	25
18.2	Output Window.	26
19	Write a C++ program to demonstrate the use of namespaces.	26
19.1	Source Code.	26
19.2	Output Window.	27
20	Write a C++ program to perform addition using a function template.	28
20.1	Source Code.	28
20.2	Output Window.	28
21	Write a C++ program to find the maximum of two values using a function template.	29
21.1	Source Code.	29
21.2	Output Window.	29
22	Write a C++ program to open a file in read mode and display its contents.	30
22.1	Source Code.	30
22.2	Output Window.	31
23	Write a C++ program to open a file in write mode and write data into it.	31

23.1	Source Code.	31
23.2	Output Window.	32
24	Write a C++ program to demonstrate random access file handling and explain how it differs from sequential access.	32
24.1	Source Code.	32
24.2	Output Window.	33
25	Write a C++ program to demonstrate exception handling using try, throw, and catch.....	33
25.1	Source Code.	33
25.2	Output Window.	34

1 Write a C++ program to input a number and check whether it is a prime number or not.

1.1 Source Code.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    bool isPrime = true;

    cout << "Enter a number: ";
    cin >> n;

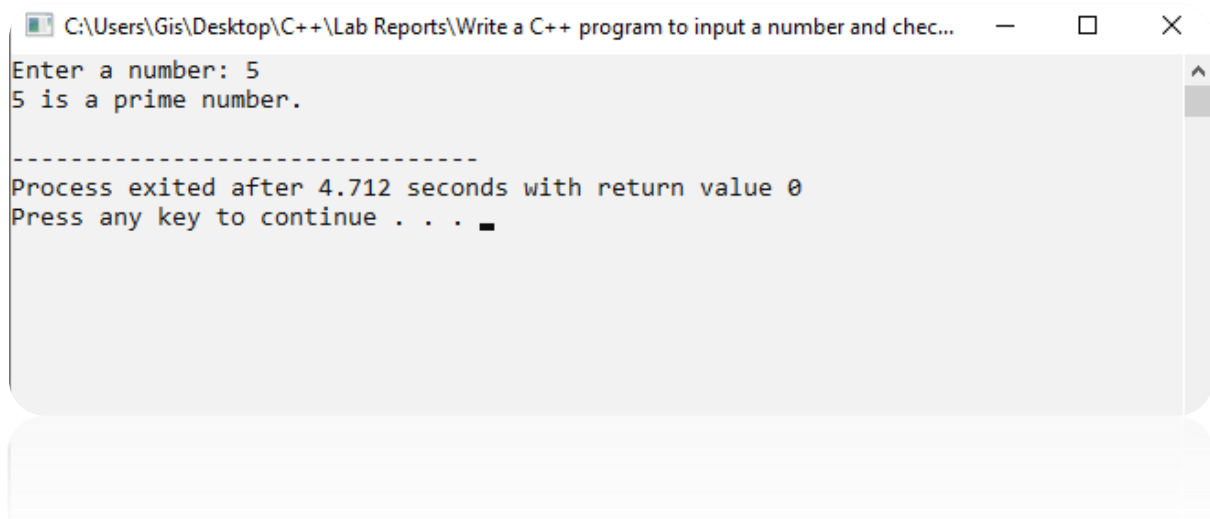
    // Numbers less than or equal to 1 are not prime
    if (n <= 1) {
        isPrime = false;
    }
    else {
        // Check whether the number is divisible by any number
        // from 2 up to n/2
        for (int i = 2; i <= n / 2; i++) {

            // If n is exactly divisible by i,
            // then n is not a prime number
            if (n % i == 0) {
                isPrime = false;
                break; // Stop checking further
            }
        }
    }

    if (isPrime) {
        cout << n << " is a prime number." << endl;
    }
    else {
        cout << n << " is not a prime number." << endl;
    }

    return 0;
}
```

1.2 Output Window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\Write a C++ program to input a number and chec...
Enter a number: 5
5 is a prime number.

-----
Process exited after 4.712 seconds with return value 0
Press any key to continue . . .
```

2 Write a C++ program to find the sum of numbers from 1 to n.

2.1 Source Code.

```
#include <iostream>
using namespace std;

int main()
{
    int n, sum = 0;

    // Input the value of n
    cout << "Enter a number: ";
    cin >> n;

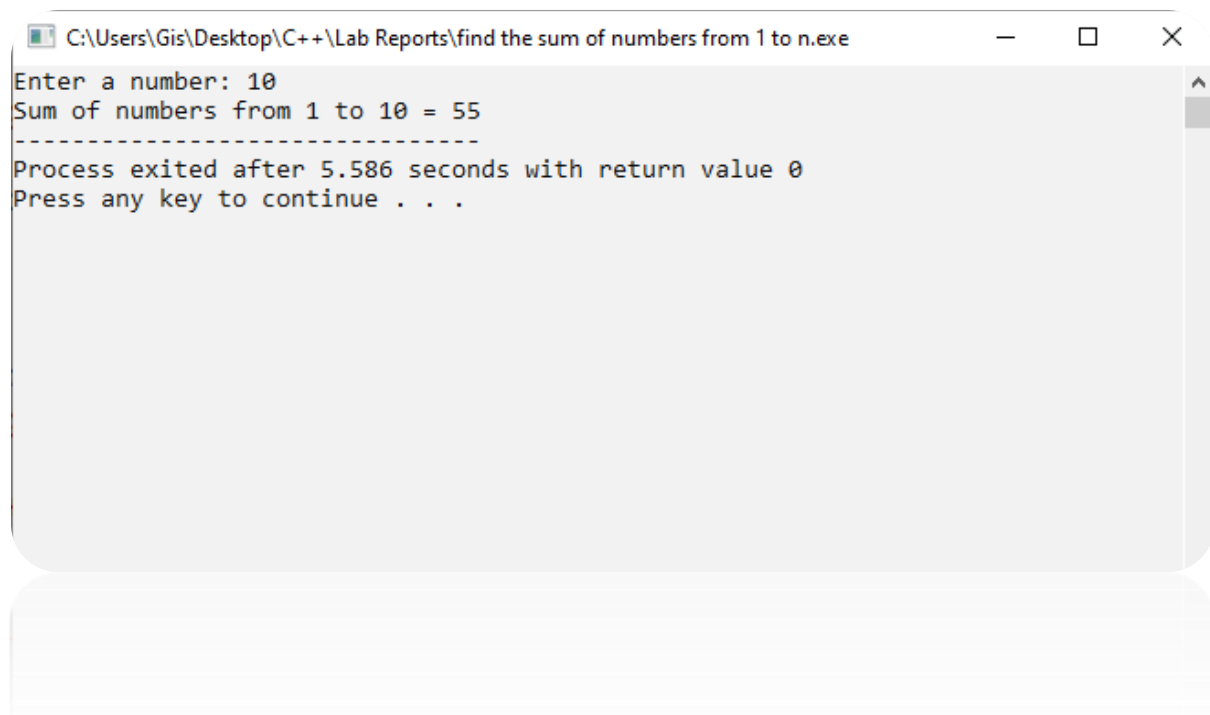
    // Find the sum of numbers from 1 to n
    for (int i = 1; i <= n; i++)
    {
        sum = sum + i;
    }

    // Display the result
    cout << "Sum of numbers from 1 to " << n << " = " << sum;

    return 0;
}
```

```
}
```

2.2 Output Window.



3 Write a C++ program to find the factorial of a number using recursion.

3.1 Source Code.

```
#include <iostream>
using namespace std;

// Function to find factorial using recursion
int factorial(int n)
{
    // Base condition
    if (n == 0 || n == 1)
        return 1;

    // Recursive call
    return n * factorial(n - 1);
}

int main()
{
```

```

    int n;

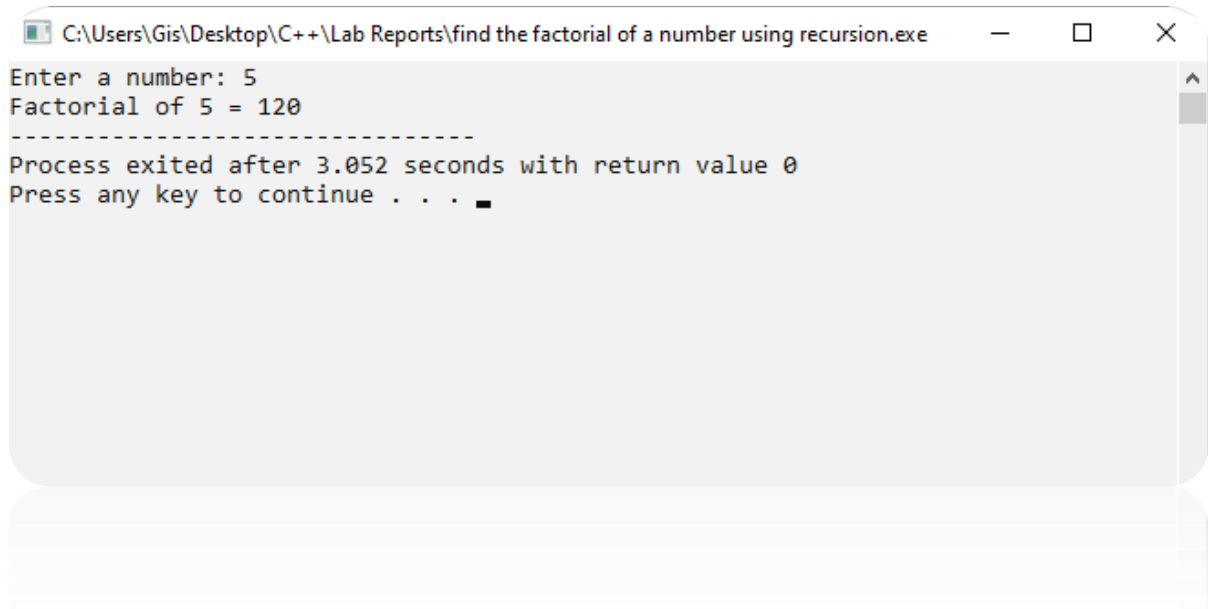
    // Input a number
    cout << "Enter a number: ";
    cin >> n;

    // Display the factorial
    cout << "Factorial of " << n << " = " << factorial(n);

    return 0;
}

```

3.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\find the factorial of a number using recursion.exe
Enter a number: 5
Factorial of 5 = 120
-----
Process exited after 3.052 seconds with return value 0
Press any key to continue . . .

```

4 Write a C++ program to calculate the area of a circle, rectangle, and square using method overloading.

4.1 Source code.

```

#include <iostream>
using namespace std;

float area(float r)
{
    return 3.14 * r * r;
}

// alculating area of rectangle
int area(int l, int b)

```

```

{
    return l * b;
}

// calculating area of square
int area(int s)
{
    return s * s;
}

int main()
{
    float r;
    int l, b, s;

    // Input radius of circle
    cout << "Enter radius of circle: ";
    cin >> r;

    // Input length and breadth of rectangle
    cout << "Enter length and breadth of rectangle: ";
    cin >> l >> b;

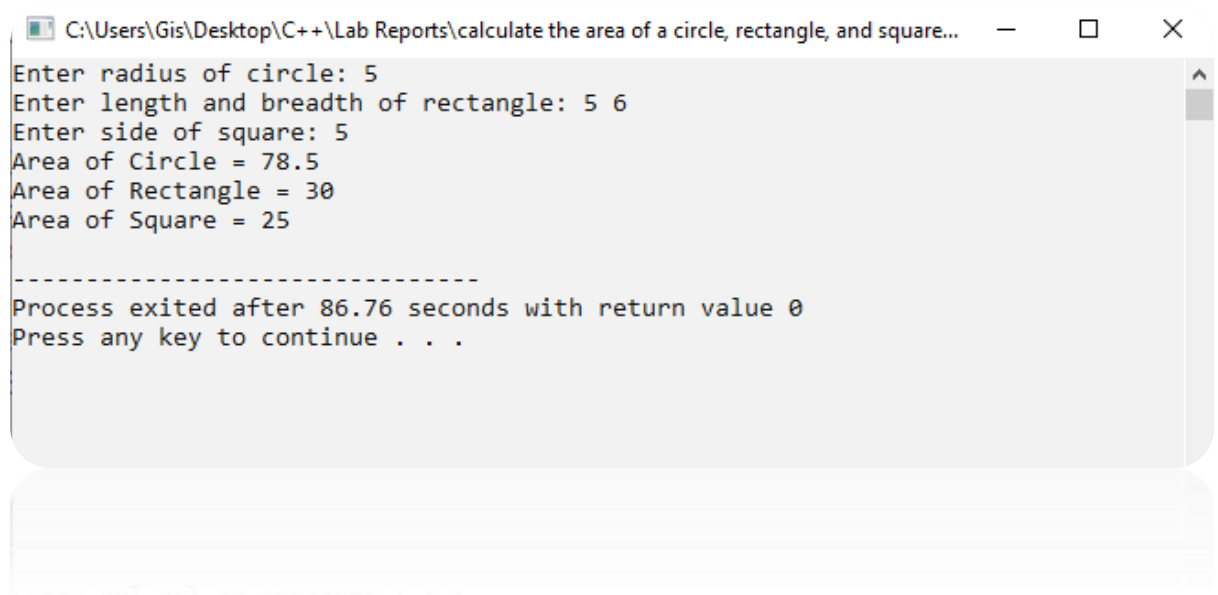
    // Input side of square
    cout << "Enter side of square: ";
    cin >> s;

    // Display areas
    cout << "Area of Circle = " << area(r) << endl;
    cout << "Area of Rectangle = " << area(l, b) << endl;
    cout << "Area of Square = " << area(s) << endl;

    return 0;
}

```

4.2 Output Window.



5 Write a C++ program to swap two numbers using pass by value.

5.1 Source Code.

```
#include <iostream>
using namespace std;

// Function to swap two numbers using pass by value
void swap(int a, int b)
{
    int temp;

    // Swap the values
    temp = a;
    a = b;
    b = temp;

    cout << "Inside function: a = " << a << ", b = " << b << endl;
}

int main()
{
    int a, b;
```

```

// Input two numbers
cout << "Enter two numbers: ";
cin >> a >> b;

// Display before swapping
cout << "Before swapping: a = " << a << ", b = " << b << endl;

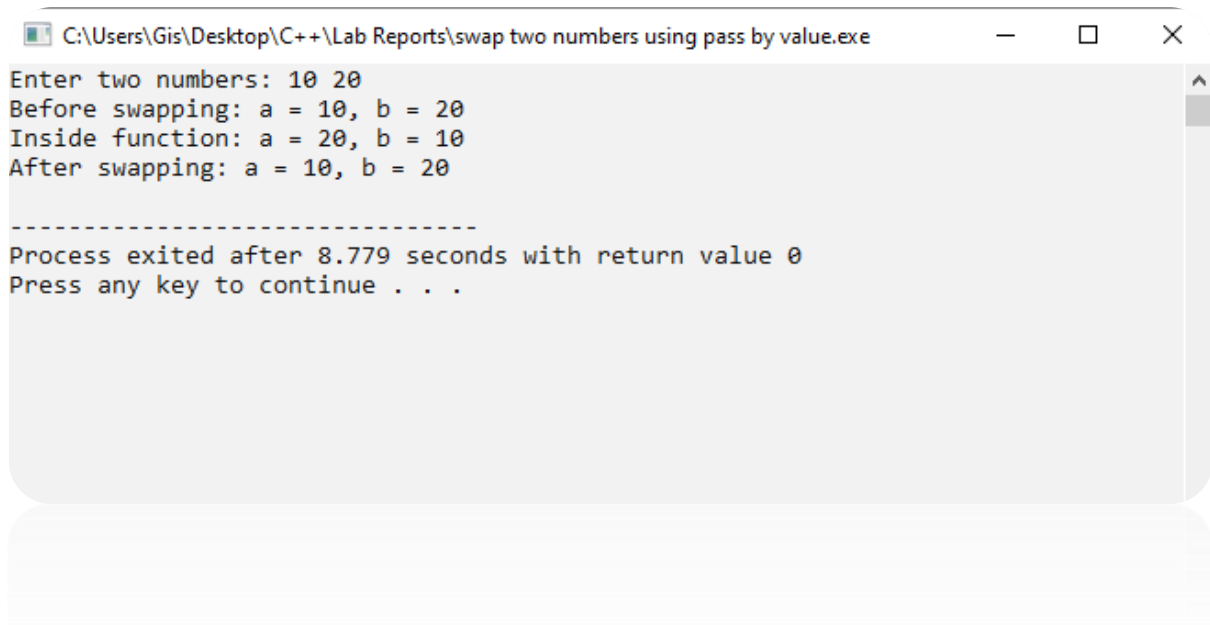
// Function call
swap(a, b);

// Display after swapping
cout << "After swapping: a = " << a << ", b = " << b << endl;

return 0;
}

```

5.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\swap two numbers using pass by value.exe
Enter two numbers: 10 20
Before swapping: a = 10, b = 20
Inside function: a = 20, b = 10
After swapping: a = 10, b = 20

-----
Process exited after 8.779 seconds with return value 0
Press any key to continue . . .

```

6 Write a C++ program to swap two numbers using pass by reference.

6.1 Source Code.

```
#include <iostream>
using namespace std;

// Function to swap two numbers using pass by reference
void swap(int &a, int &b)
{
    int temp;

    // Swap the values
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    int a, b;

    // Input two numbers
    cout << "Enter two numbers: ";
    cin >> a >> b;

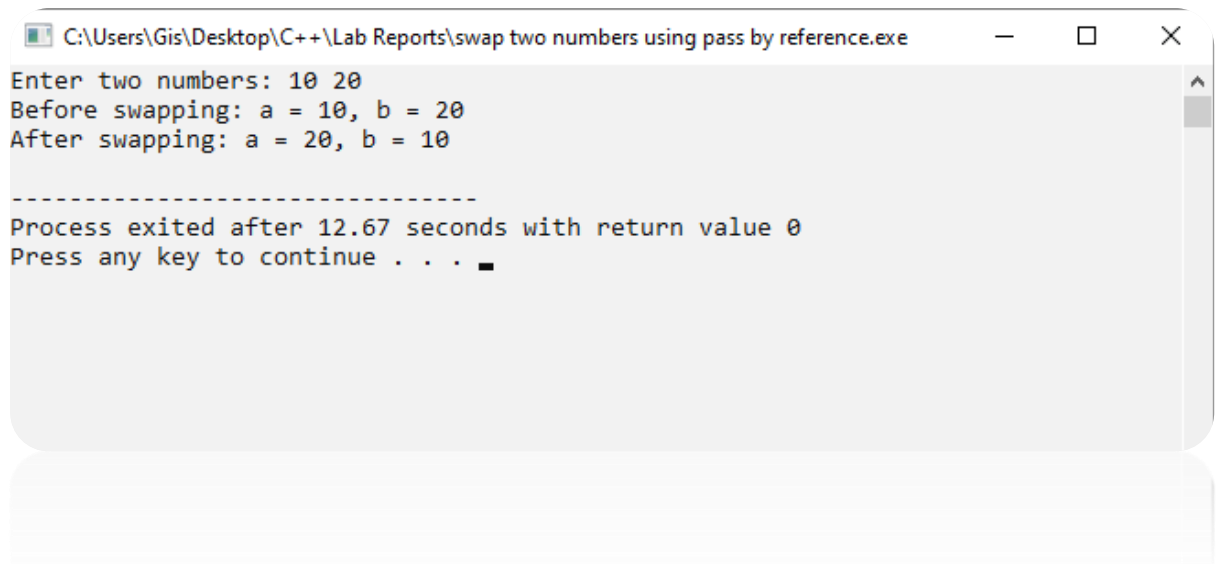
    // Display values before swapping
    cout << "Before swapping: a = " << a << ", b = " << b << endl;

    // Call the function
    swap(a, b);

    // Display values after swapping
    cout << "After swapping: a = " << a << ", b = " << b << endl;

    return 0;
}
```

6.2 Output Window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\swap two numbers using pass by reference.exe
Enter two numbers: 10 20
Before swapping: a = 10, b = 20
After swapping: a = 20, b = 10
-----
Process exited after 12.67 seconds with return value 0
Press any key to continue . . .
```

7 Write a C++ program to swap two numbers without using a third variable.

7.1 Source Code.

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    // Input two numbers
    cout << "Enter two numbers: ";
    cin >> a >> b;

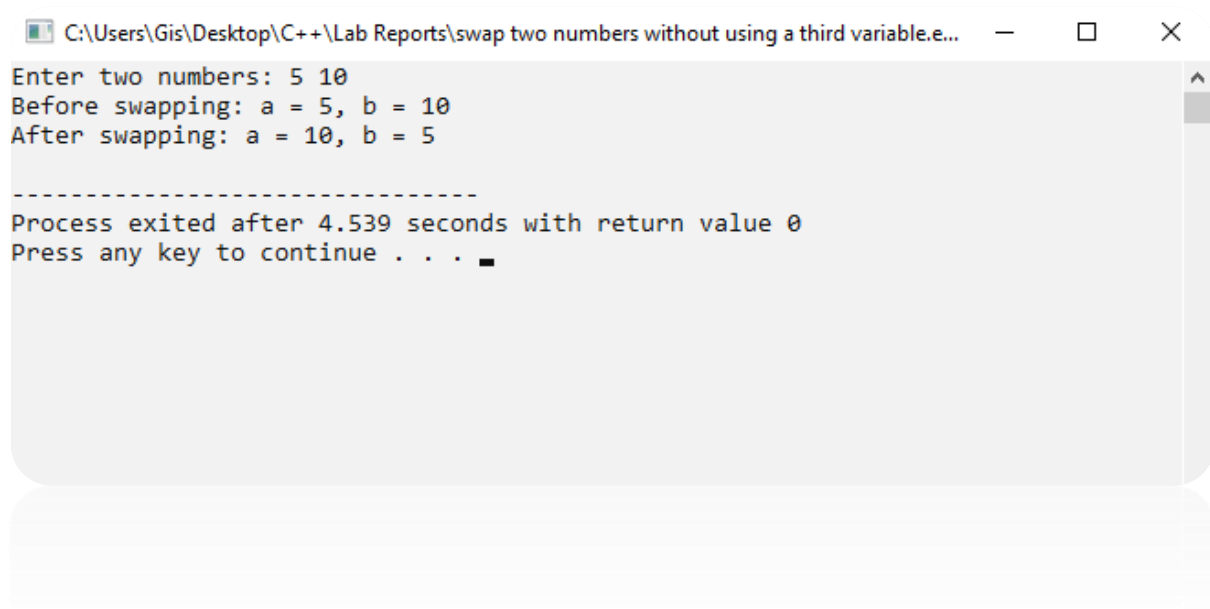
    // Display numbers before swapping
    cout << "Before swapping: a = " << a << ", b = " << b << endl;

    // Swap without using a third variable
    a = a + b;
    b = a - b;
    a = a - b;

    // Display numbers after swapping
    cout << "After swapping: a = " << a << ", b = " << b << endl;

    return 0;
}
```

7.2 Output Window.



The screenshot shows a Windows-style window titled "C:\Users\Gis\Desktop\C++\Lab Reports\swap two numbers without using a third variable.e...". The window contains the following text:

```
Enter two numbers: 5 10
Before swapping: a = 5, b = 10
After swapping: a = 10, b = 5

-----
Process exited after 4.539 seconds with return value 0
Press any key to continue . . . █
```

8 Write a C++ program to input a number and print its reverse.

8.1 Source Code.

```
#include <iostream>
using namespace std;

int main()
{
    int n, reverse = 0, remainder;

    // Input a number
    cout << "Enter a number: ";
    cin >> n;

    // Find the reverse of the number
    while (n != 0)
    {
        // Get the last digit
        remainder = n % 10;

        // Add the digit to the reverse
        reverse = reverse * 10 + remainder;

        // Remove the last digit
        n = n / 10;
    }
}
```

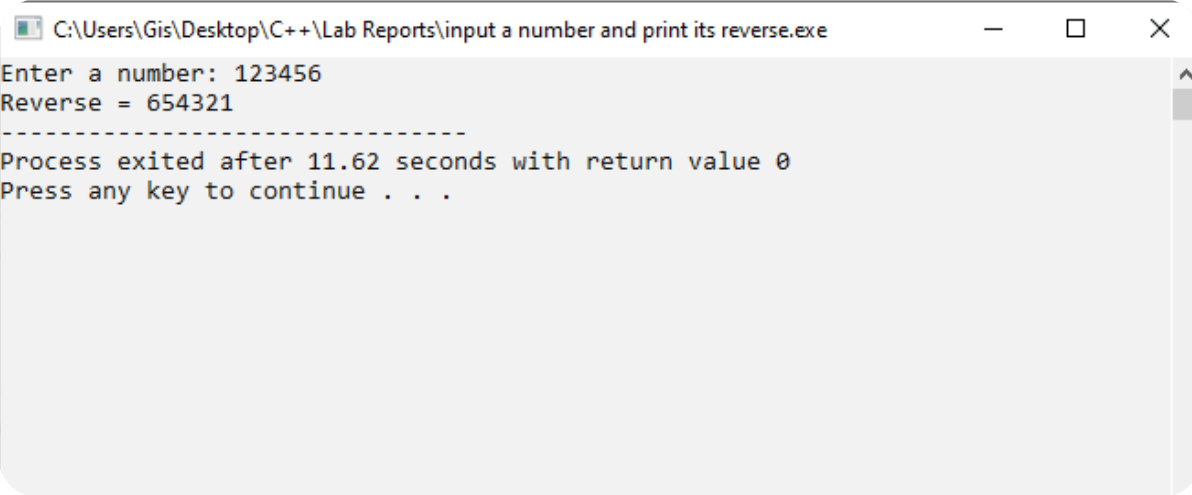
```

    // Display the reversed number
    cout << "Reverse = " << reverse;

    return 0;
}

```

8.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\input a number and print its reverse.exe
Enter a number: 123456
Reverse = 654321
-----
Process exited after 11.62 seconds with return value 0
Press any key to continue . . .

```

9 Write a C++ program to demonstrate constructor overloading.

9.1 Source Code.

```

#include <iostream>
using namespace std;

class Student
{
    int roll;
    string name;

public:

    // Default constructor
    Student()
    {
        roll = 0;
        name = "Unknown";
    }
}

```

```

// Parameterized constructor with one argument
Student(int r)
{
    roll = r;
    name = "Unknown";
}

// Parameterized constructor with two arguments
Student(int r, string n)
{
    roll = r;
    name = n;
}

// Function to display student details
void display()
{
    cout << "Roll No: " << roll << endl;
    cout << "Name: " << name << endl;
}
};

int main()
{
    // Calling default constructor
    Student s1;

    // Calling constructor with one argument
    Student s2(101);

    // Calling constructor with two arguments
    Student s3(102, "Ram");

    cout << "Student 1:" << endl;
    s1.display();

    cout << "\nStudent 2:" << endl;
    s2.display();

    cout << "\nStudent 3:" << endl;
    s3.display();

    return 0;
}

```

9.2 Output window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate constructor overloading.exe

Student 1:
Roll No: 0
Name: Unknown

Student 2:
Roll No: 101
Name: Unknown

Student 3:
Roll No: 102
Name: Ram

-----
Process exited after 0.1761 seconds with return value 0
Press any key to continue . . .
```

10 Write a C++ program to demonstrate encapsulation.

10.1 Source Code.

```
#include <iostream>
using namespace std;

class Student
{
private:
    // Private data members
    int rollNo;
    string name;

public:
    // Setter function to set values
    void setData(int r, string n)
    {
        rollNo = r;
        name = n;
    }

    // Getter function to display values
    void getData()
    {
        cout << "Roll No: " << rollNo << endl;
```

```

        cout << "Name: " << name << endl;
    }
};

int main()
{
    Student s;

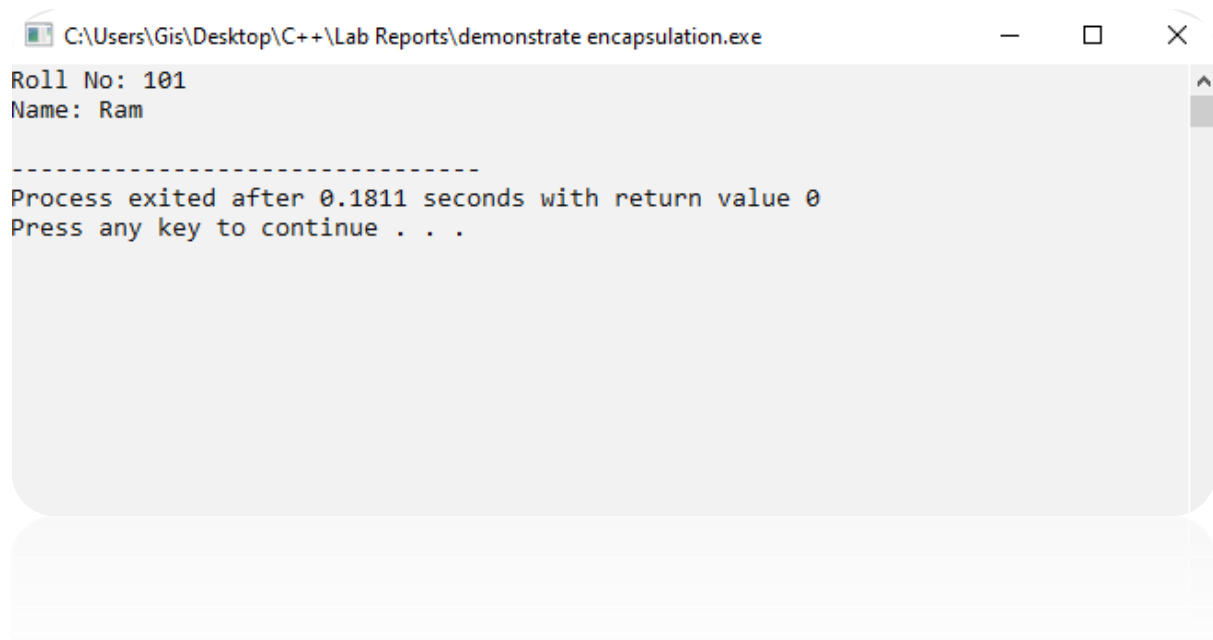
    // Set student data
    s.setData(101, "Ram");

    // Display student data
    s.getData();

    return 0;
}

```

10.2 Output window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate encapsulation.exe
Roll No: 101
Name: Ram
-----
Process exited after 0.1811 seconds with return value 0
Press any key to continue . . .

```

11 Write a C++ program to demonstrate a friend function.

11.1 Source Code.

```

#include <iostream>
using namespace std;

class Student
{
private:
    int marks;

```

```

public:
    // Function to set marks
    void setMarks(int m)
    {
        marks = m;
    }

    // Declare friend function
    friend void displayMarks(Student s);
};

// Friend function
void displayMarks(Student s)
{
    // Friend function can access private data
    cout << "Marks = " << s.marks << endl;
}

int main()
{
    Student s;

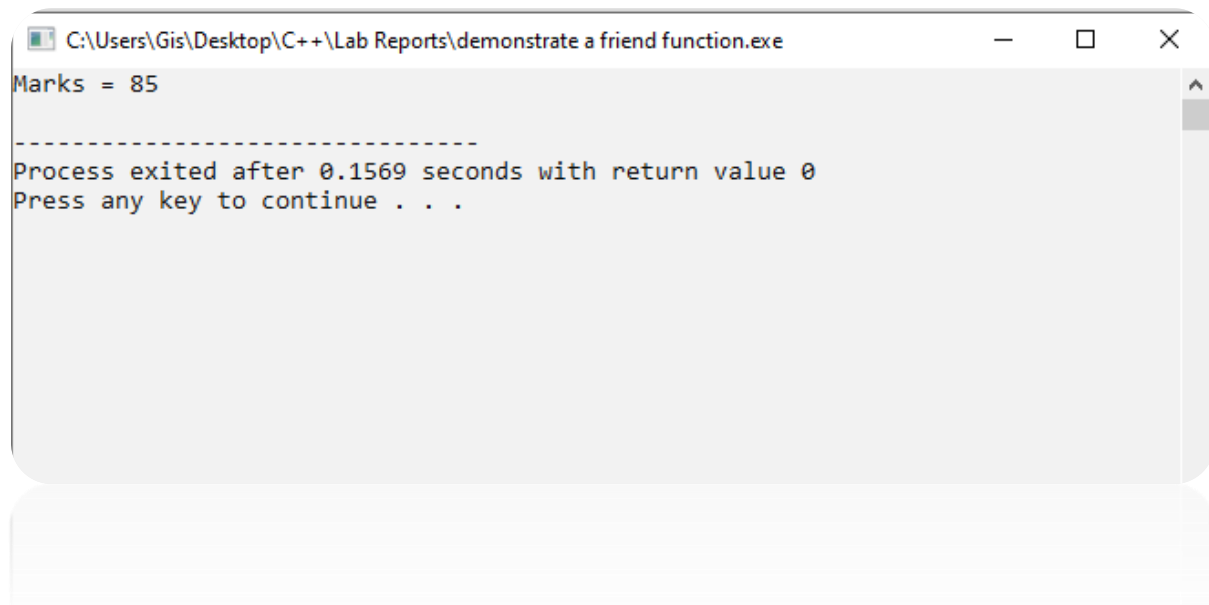
    // Set marks
    s.setMarks(85);

    // Call friend function
    displayMarks(s);

    return 0;
}

```

11.2 Output Window.



12 Write a C++ program to demonstrate a static data member.

12.1 Source Code.

```
#include <iostream>
using namespace std;

class Student
{
private:
    int rollNo;

public:
    // Static data member
    static int count;

    // Constructor
    Student(int r)
    {
        rollNo = r;
        count++;
    }

    // Function to display roll number
    void display()
    {
```

```

        cout << "Roll No: " << rollNo << endl;
    }
};

// Initialize static data member
int Student::count = 0;

int main()
{
    // Create objects
    Student s1(101);
    Student s2(102);
    Student s3(103);

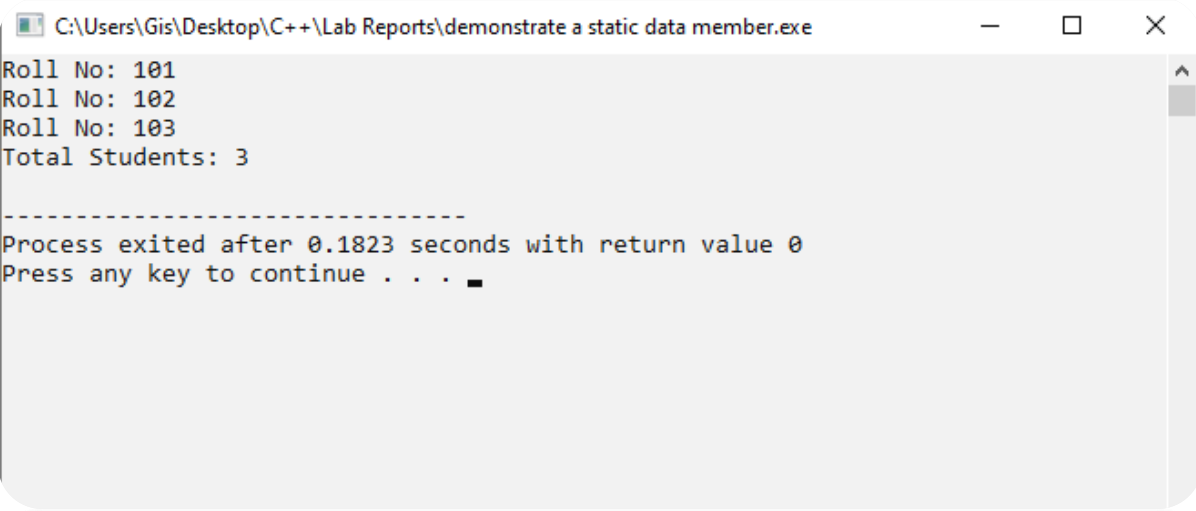
    // Display student details
    s1.display();
    s2.display();
    s3.display();

    // Display number of objects created
    cout << "Total Students: " << Student::count << endl;

    return 0;
}

```

12.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate a static data member.exe
Roll No: 101
Roll No: 102
Roll No: 103
Total Students: 3

-----
Process exited after 0.1823 seconds with return value 0
Press any key to continue . . .

```

13 Write a C++ program to demonstrate the execution sequence of constructors in base and derived classes.

13.1 Source code.

```
#include <iostream>
using namespace std;

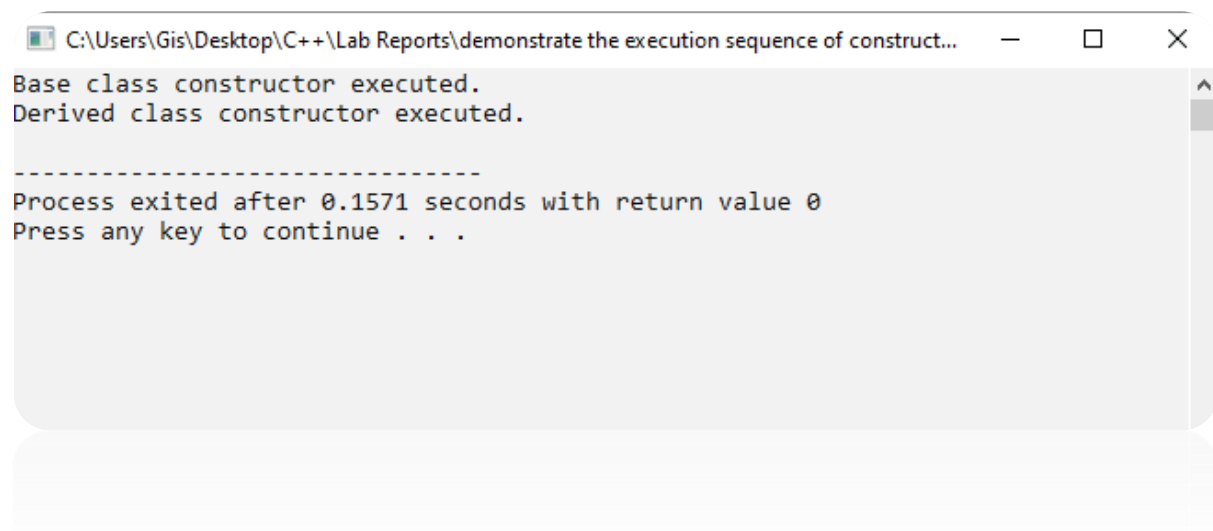
// Base class
class Base
{
public:
    // Base class constructor
    Base()
    {
        cout << "Base class constructor executed." << endl;
    }
};

// Derived class
class Derived : public Base
{
public:
    // Derived class constructor
    Derived()
    {
        cout << "Derived class constructor executed." << endl;
    }
};

int main()
{
    // Create an object of derived class
    Derived obj;

    return 0;
}
```

13.2 Output Window.

A screenshot of a Windows console window. The title bar shows the file path: C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate the execution sequence of construct... The window contains the following text: "Base class constructor executed.", "Derived class constructor executed.", a dashed line separator, "Process exited after 0.1571 seconds with return value 0", and "Press any key to continue . . .".

```
C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate the execution sequence of construct...
Base class constructor executed.
Derived class constructor executed.
-----
Process exited after 0.1571 seconds with return value 0
Press any key to continue . . .
```

14 Write a C++ program to demonstrate different modes of inheritance.

14.1 Source Code.

```
#include <iostream>
using namespace std;

class Base
{
public:
    int a = 10;

protected:
    int b = 20;

private:
    int c = 30;
};

// Public inheritance
class PublicDerived : public Base
{
public:
    void display()
    {
        cout << "Public Inheritance: " << a << " " << b << endl;
    }
};
```

```

// Protected inheritance
class ProtectedDerived : protected Base
{
public:
    void display()
    {
        cout << "Protected Inheritance: " << a << " " << b << endl;
    }
};

// Private inheritance
class PrivateDerived : private Base
{
public:
    void display()
    {
        cout << "Private Inheritance: " << a << " " << b << endl;
    }
};

int main()
{
    PublicDerived obj1;
    ProtectedDerived obj2;
    PrivateDerived obj3;

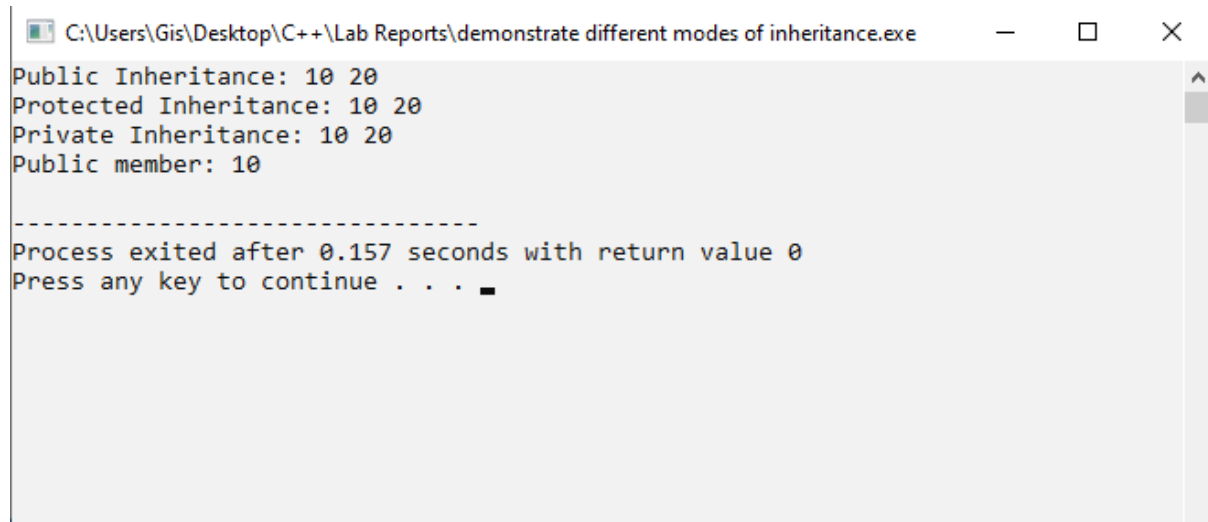
    // Display values
    obj1.display();
    obj2.display();
    obj3.display();

    // Public member can be accessed directly
    cout << "Public member: " << obj1.a << endl;

    return 0;
}

```

14.2 Output Window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate different modes of inheritance.exe
Public Inheritance: 10 20
Protected Inheritance: 10 20
Private Inheritance: 10 20
Public member: 10

-----
Process exited after 0.157 seconds with return value 0
Press any key to continue . . . █
```

15 Write a C++ program to demonstrate overriding methods.

15.1 Source Code.

```
#include <iostream>
using namespace std;

// Base class
class Animal
{
public:
    // Method in base class
    void sound()
    {
        cout << "Animal makes a sound." << endl;
    }
};

// Derived class
class Dog : public Animal
{
public:
    // Overriding the sound() method
    void sound()
    {
        cout << "Dog barks." << endl;
    }
};
```

```

int main()
{
    Animal a;
    Dog d;

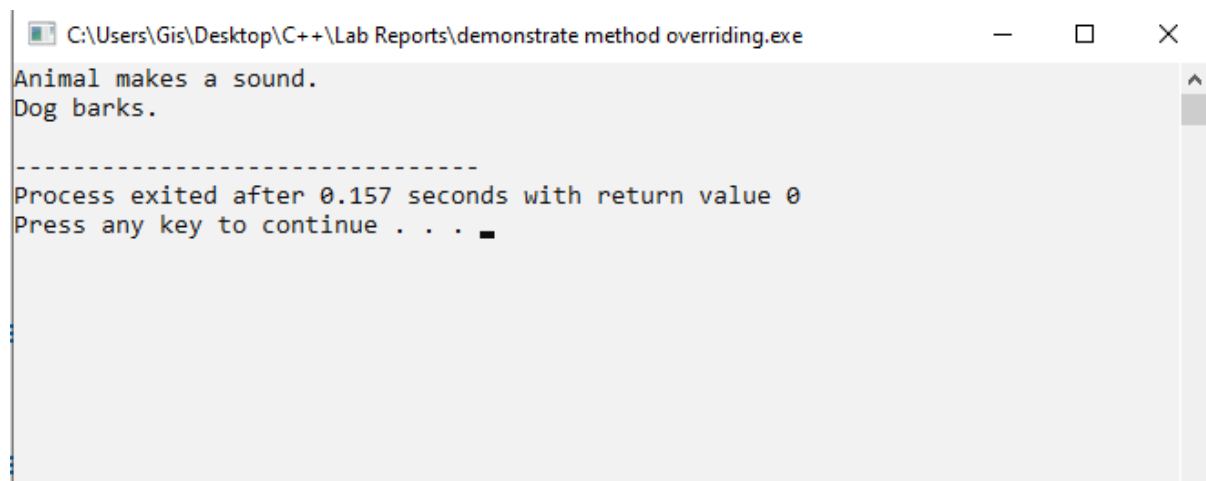
    // Call base class method
    a.sound();

    // Call overridden method
    d.sound();

    return 0;
}

```

15.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate method overriding.exe
Animal makes a sound.
Dog barks.

-----
Process exited after 0.157 seconds with return value 0
Press any key to continue . . .

```

16 Write a C++ program to demonstrate arithmetic operator overloading.

16.1 Source Code.

```

#include <iostream>
using namespace std;

class Number
{
private:
    int value;

public:
    // Constructor
    Number(int v)
    {
        value = v;
    }
}

```

```

    }

    // Overload + operator
    Number operator+(Number n)
    {
        return Number(value + n.value);
    }

    // Function to display value
    void display()
    {
        cout << "Result = " << value << endl;
    }
};

int main()
{
    // Create two objects
    Number n1(10);
    Number n2(20);

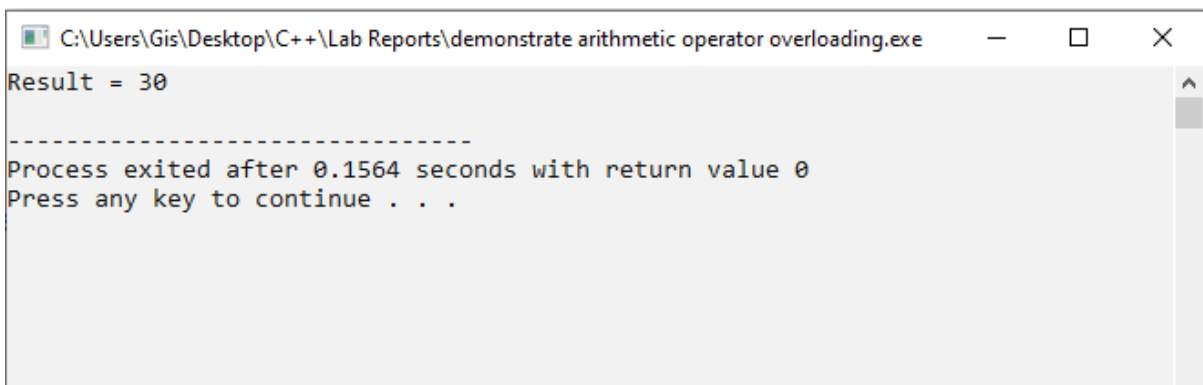
    // Add two objects using overloaded + operator
    Number n3 = n1 + n2;

    // Display result
    n3.display();

    return 0;
}

```

16.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate arithmetic operator overloading.exe
Result = 30
-----
Process exited after 0.1564 seconds with return value 0
Press any key to continue . . .

```

17 Write a C++ program to demonstrate relational operator overloading.

17.1 Source Code.

```
#include <iostream>
using namespace std;

class Number
{
private:
    int value;

public:
    // Constructor
    Number(int v)
    {
        value = v;
    }

    // Overload + operator
    Number operator+(Number n)
    {
        return Number(value + n.value);
    }

    // Function to display value
    void display()
    {
        cout << "Result = " << value << endl;
    }
};

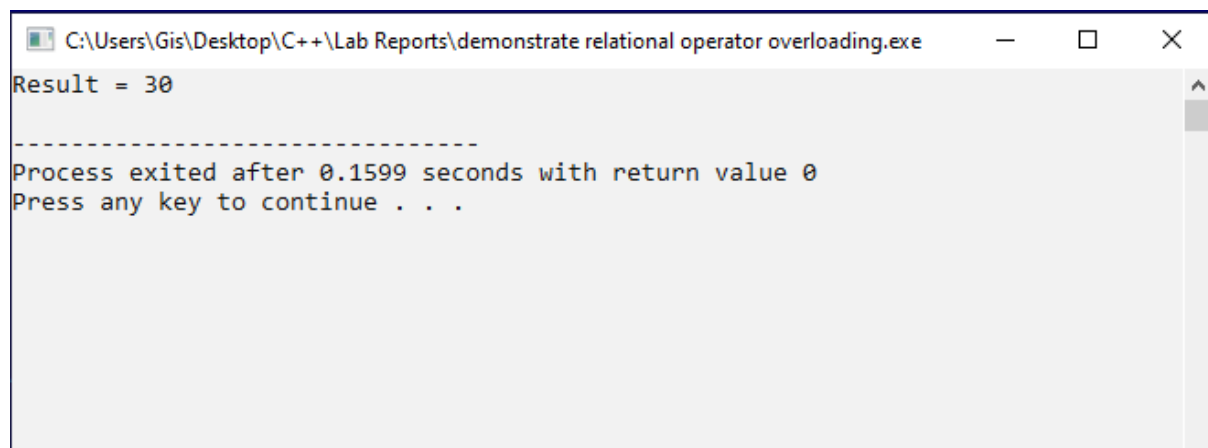
int main()
{
    // Create two objects
    Number n1(10);
    Number n2(20);

    // Add two objects using overloaded + operator
    Number n3 = n1 + n2;

    // Display result
    n3.display();

    return 0;}
```

17.2 Output Window.



18 Write a C++ program to demonstrate unary operator overloading.

18.1 Source Code.

```
#include <iostream>
using namespace std;

class Number
{
private:
    int value;

public:
    // Constructor
    Number(int v)
    {
        value = v;
    }

    // Overload unary ++ operator
    void operator++()
    {
        ++value;
    }

    // Function to display value
    void display()
    {
        cout << "Value = " << value << endl;
    }
};
```

```

int main()
{
    // Create an object
    Number n(10);

    // Display original value
    cout << "Before increment:" << endl;
    n.display();

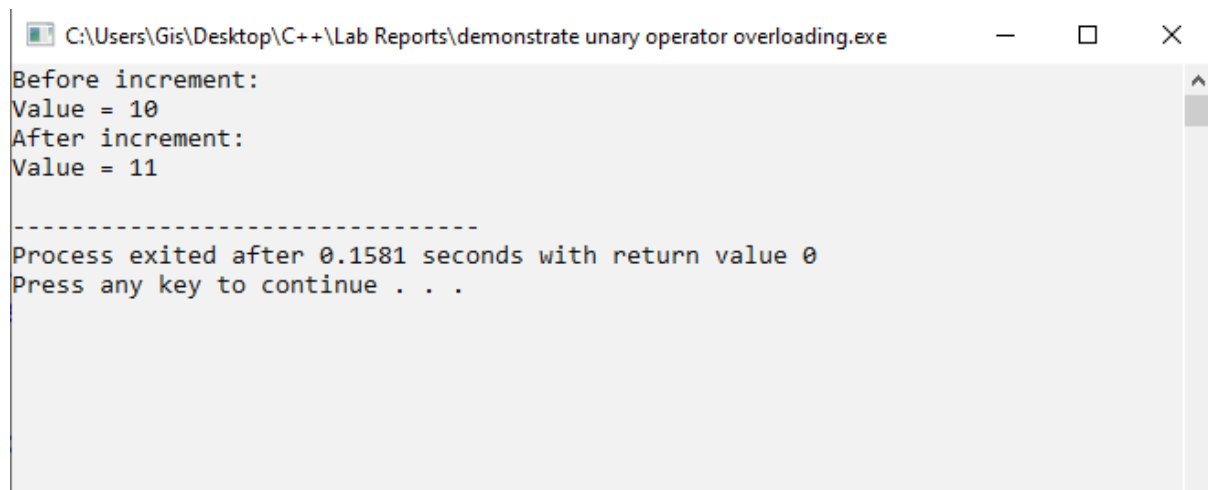
    // Call overloaded ++ operator
    ++n;

    // Display value after increment
    cout << "After increment:" << endl;
    n.display();

    return 0;
}

```

18.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate unary operator overloading.exe
Before increment:
Value = 10
After increment:
Value = 11

-----
Process exited after 0.1581 seconds with return value 0
Press any key to continue . . .

```

19 Write a C++ program to demonstrate the use of namespaces.

19.1 Source Code.

```

#include <iostream>
using namespace std;

// First namespace
namespace First
{
    int value = 10;
}

```

```

    void display()
    {
        cout << "Value from First namespace = " << value << endl;
    }
}

// Second namespace
namespace Second
{
    int value = 20;

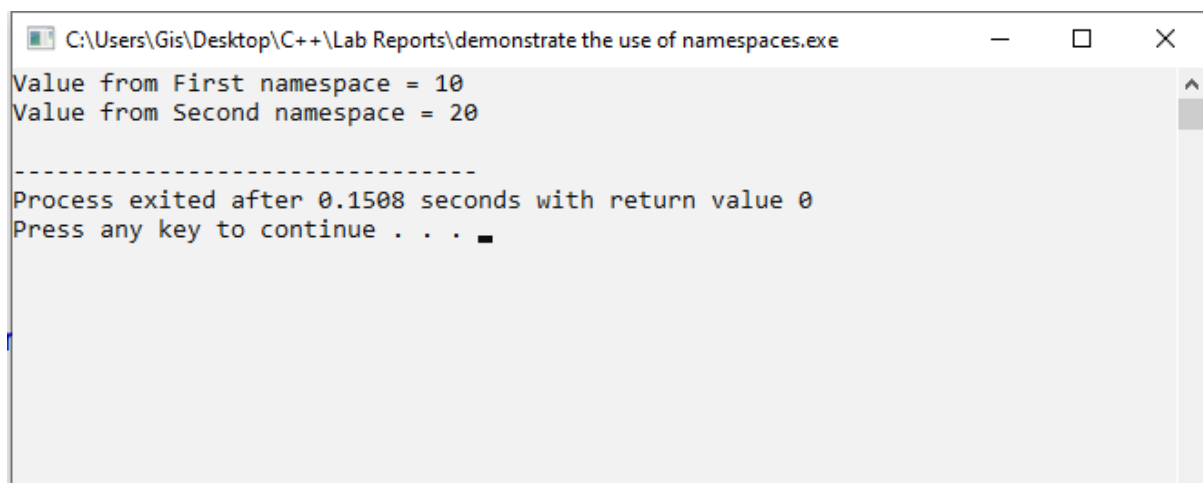
    void display()
    {
        cout << "Value from Second namespace = " << value << endl;
    }
}

int main()
{
    // Access members using namespace name
    First::display();
    Second::display();

    return 0;
}

```

19.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate the use of namespaces.exe
Value from First namespace = 10
Value from Second namespace = 20

-----
Process exited after 0.1508 seconds with return value 0
Press any key to continue . . .

```

20 Write a C++ program to perform addition using a function template.

20.1 Source Code.

```
#include <iostream>
using namespace std;

// Function template for addition
template <class T>
T add(T a, T b)
{
    return a + b;
}

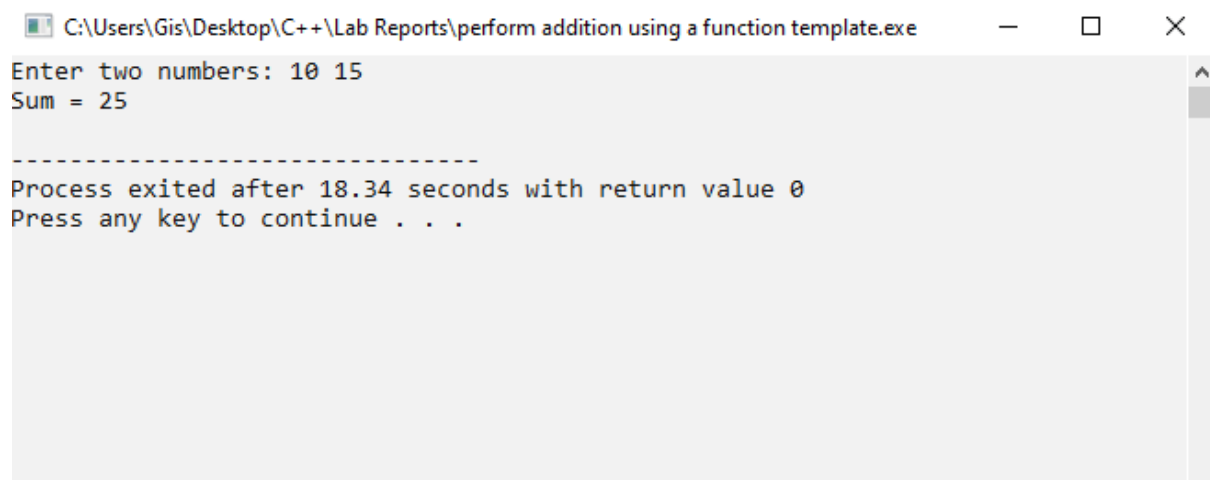
int main()
{
    int a, b;

    // Input two numbers
    cout << "Enter two numbers: ";
    cin >> a >> b;

    // Call function template
    cout << "Sum = " << add(a, b) << endl;

    return 0;
}
```

20.2 Output Window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\perform addition using a function template.exe
Enter two numbers: 10 15
Sum = 25

-----
Process exited after 18.34 seconds with return value 0
Press any key to continue . . .
```

21 Write a C++ program to find the maximum of two values using a function template.

21.1 Source Code.

```
#include <iostream>
using namespace std;

// Function template to find maximum
template <class T>
T maximum(T a, T b)
{
    // Compare two values
    if (a > b)
        return a;
    else
        return b;
}

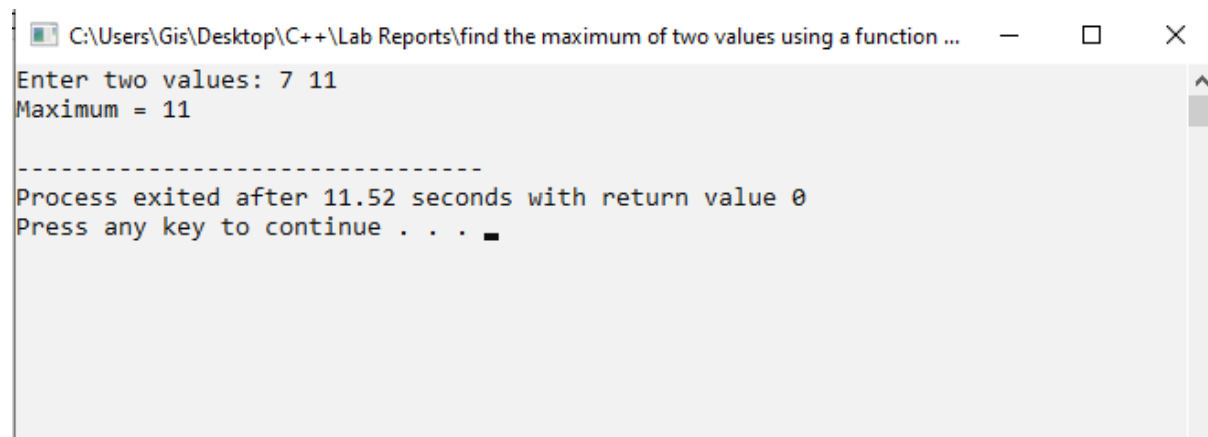
int main()
{
    int a, b;

    // Input two values
    cout << "Enter two values: ";
    cin >> a >> b;

    // Display maximum value
    cout << "Maximum = " << maximum(a, b) << endl;

    return 0;
}
```

21.2 Output Window.



```
C:\Users\Gis\Desktop\C++\Lab Reports\find the maximum of two values using a function ...
Enter two values: 7 11
Maximum = 11

-----
Process exited after 11.52 seconds with return value 0
Press any key to continue . . .
```

22 Write a C++ program to open a file in read mode and display its contents.

22.1 Source Code.

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    string line;

    // Open the file in read mode
    ifstream file("data.txt");

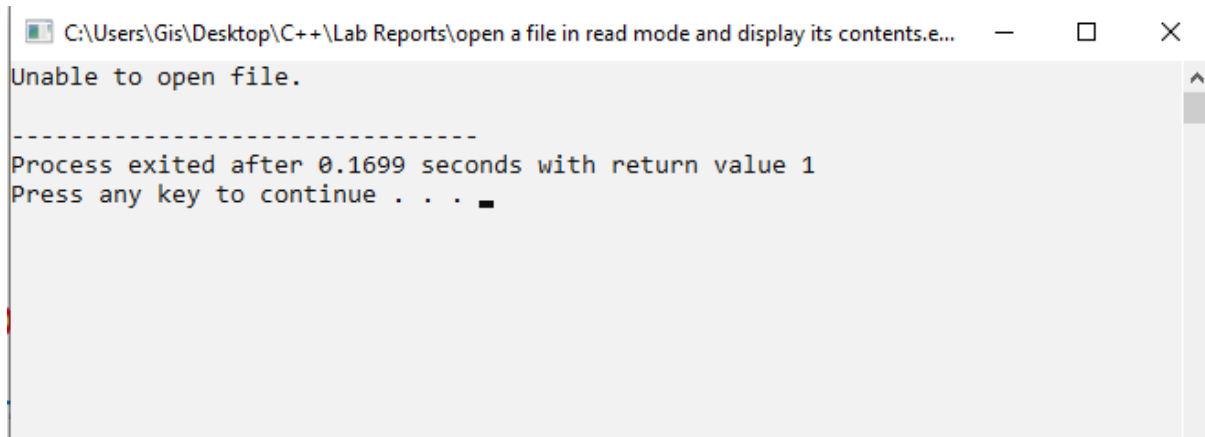
    // Check if the file opened successfully
    if (!file)
    {
        cout << "Unable to open file." << endl;
        return 1;
    }

    // Read the file line by line
    while (getline(file, line))
    {
        // Display each line
        cout << line << endl;
    }

    // Close the file
    file.close();

    return 0;
}
```

22.2 Output Window.



23 Write a C++ program to open a file in write mode and write data into it.

23.1 Source Code.

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    // Open file in write mode
    ofstream file("data.txt");

    // Check if the file opened successfully
    if (!file)
    {
        cout << "Unable to open file." << endl;
        return 1;
    }

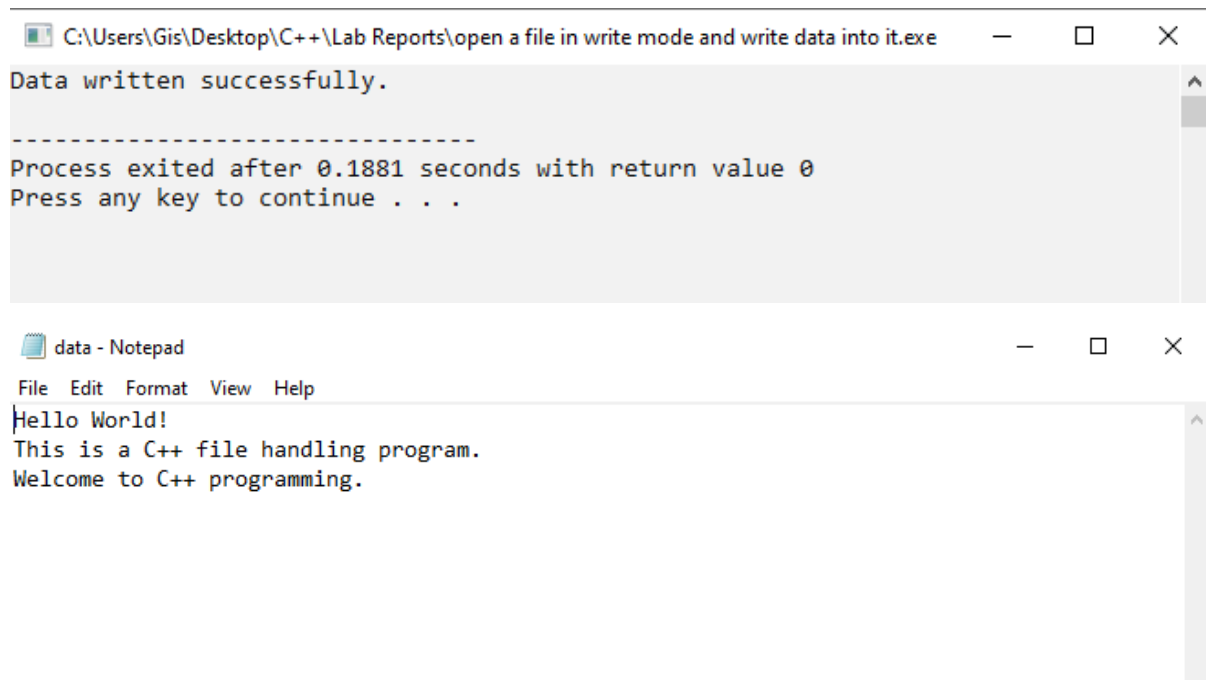
    // Write data into the file
    file << "Hello World!" << endl;
    file << "This is a C++ file handling program." << endl;
    file << "Welcome to C++ programming." << endl;

    // Close the file
    file.close();

    cout << "Data written successfully." << endl;

    return 0;
}
```

23.2 Output Window.



24 Write a C++ program to demonstrate random access file handling and explain how it differs from sequential access.

24.1 Source Code.

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    // Open file in read and write mode
    fstream file("data.txt", ios::in | ios::out);

    // Check if file opened successfully
    if (!file)
    {
        cout << "Unable to open file." << endl;
        return 1;
    }

    // Move the file pointer to the beginning
    file.seekg(0);

    // Read data from a specific position
```

```

file.seekg(6);

char ch;
file.get(ch);

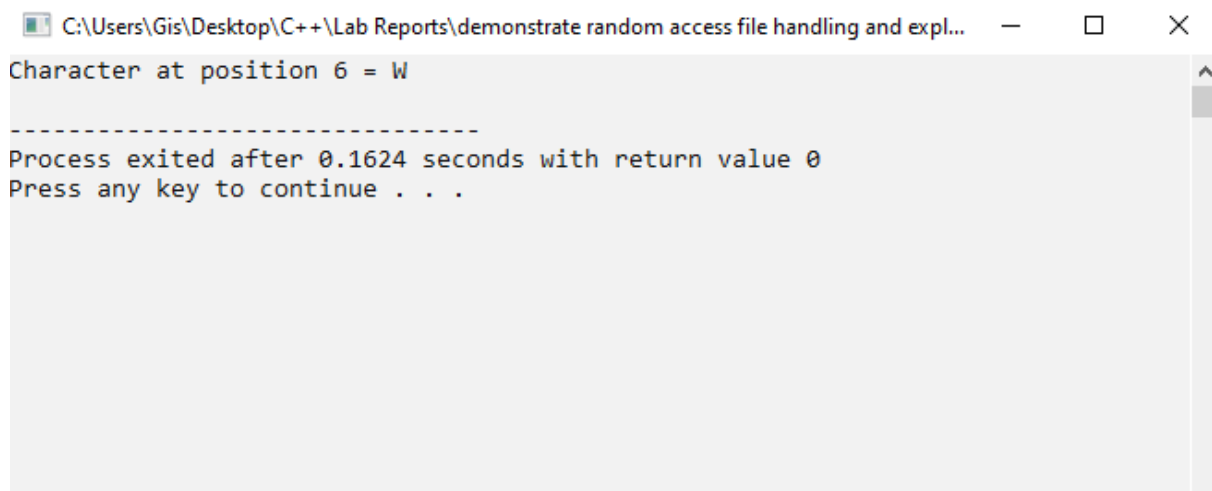
// Display the character at position 6
cout << "Character at position 6 = " << ch << endl;

// Close the file
file.close();

return 0;
}

```

24.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate random access file handling and expl...
Character at position 6 = W
-----
Process exited after 0.1624 seconds with return value 0
Press any key to continue . . .

```

25 Write a C++ program to demonstrate exception handling using try, throw, and catch.

25.1 Source Code.

```

#include <iostream>
using namespace std;

int main()
{
    int a, b;

    // Input two numbers
    cout << "Enter two numbers: ";
    cin >> a >> b;

    try

```

```

{
    // Check for division by zero
    if (b == 0)
    {
        // Throw an exception
        throw b;
    }

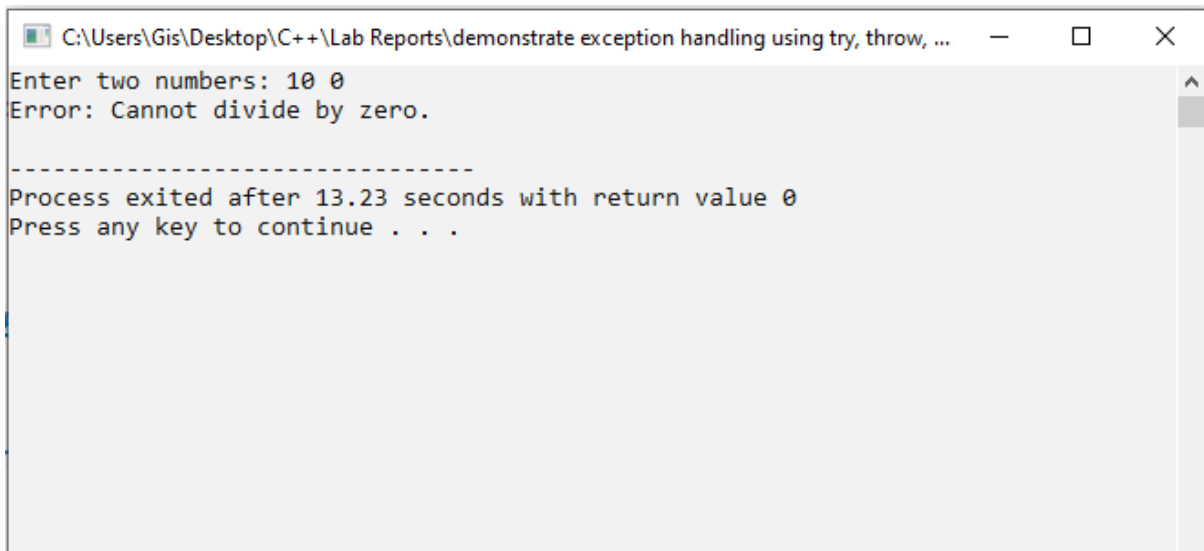
    cout << "Result = " << a / b << endl;
}

catch (int x)
{
    // Handle the exception
    cout << "Error: Cannot divide by zero." << endl;
}

return 0;
}

```

25.2 Output Window.



```

C:\Users\Gis\Desktop\C++\Lab Reports\demonstrate exception handling using try, throw, ...
Enter two numbers: 10 0
Error: Cannot divide by zero.

-----
Process exited after 13.23 seconds with return value 0
Press any key to continue . . .

```